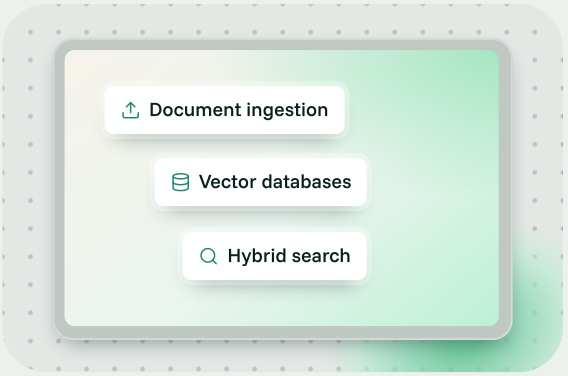
Last updated August 2026

Retrieval-Augmented Generation

How grounding language models in external knowledge reduces hallucination, enables domain-specific answers, and keeps responses current.



15 min read · Beginner-friendly with advanced sections

Pascal Academy
AI fundamentals · Part 3 of 10

What Is RAG?

In the first guide, we saw that a large language model is a probability engine trained on trillions of tokens of text. In the second, we learned how to steer that engine with prompts. Retrieval-augmented generation, or RAG, addresses a remaining limitation. The model only knows what was in its training data, and that data has a cutoff date.

RAG works by retrieving relevant documents from an external knowledge source at query time, then including those documents in the prompt so the model can ground its response in them. Instead of relying on what the model memorised during training, you give it a fresh set of facts to work from for each question. The model still does the generation, but its answer is anchored to retrieved text rather than to its own internal knowledge.

A better way to think about it is as an open-book exam. Without RAG, the model answers from memory, and memory can be incomplete, outdated, or wrong. With RAG, the model gets a set of reference materials handed to it at the moment of the question, and it constructs its answer from those materials. The quality of the answer depends on whether you handed it the right pages.

Key principle

RAG separates knowledge from reasoning. The LLM provides the reasoning, the retrieval system provides the knowledge. This separation matters because it means you can update what the model knows without retraining it. Add a document to your knowledge base and the model can answer questions about it immediately.

Why RAG Exists: The Knowledge Problem

A language model's knowledge is frozen at the moment its training ended. If the model finished training in January 2025, it has no knowledge of events after that date. It cannot tell you about a policy change announced in March, a product released in June, or a research paper published last week. The training data is also general. The model has read the public internet, but it has no access to your company's internal documents, your customer database, or your product specifications.

You could solve this by fine-tuning the model on your data, but fine-tuning is expensive, slow, and has a problematic side effect. The model's updates are again frozen. Every time your data changes, you would need to retrain. For a company whose product catalogue changes weekly, or a support system whose documentation updates daily, fine-tuning on each change is impractical.

RAG offers a different approach. Rather than baking knowledge into the model's weights, you keep the knowledge in an external database and retrieve the relevant pieces at query time. The model's weights stay fixed. The knowledge base changes independently. Add a new document, delete an old one, update a policy, and the next query will reflect those changes with no retraining required.

The Hallucination Connection

In the first guide, we discussed hallucination, models producing text that sounds confident and specific but is factually wrong. Hallucination happens because the model is optimised for plausible-sounding text, and sometimes what is plausible is also incorrect. When you ask a model about something it barely encountered in training, it has weak signal to work from, and the output degrades.

RAG reduces hallucination by giving the model strong signal, the actual text it should base its answer on. If a user asks about your company's refund policy, the model does not need to guess based on what it learned about consumer retail during pre-training. The retrieval system finds the refund policy document, hands it to the model, and the model extracts the answer from it. The model is doing extraction and synthesis rather than recall, and extraction is a task LLMs are genuinely good at.

RAG does not eliminate hallucination entirely. A model can still misread a retrieved document, conflate details from different sources, or invent information when the retrieved text does not contain the answer. The risk is lower, though, because the model has something concrete to anchor to. You can also instruct the model to say it does not know when the retrieved documents lack the answer, which further reduces fabricated responses.

The RAG Pipeline

A RAG system has two phases. An ingestion phase that happens before any questions are asked, and a query phase that happens when a user asks something. Understanding both phases is essential because failures in either one produce bad answers.

Query and Generation Steps

1. Query Embedding: Convert the user's question into a vector using the same embedding model from ingestion. The question and the documents must live in the same vector space for similarity to be meaningful.

2. Retrieval: Search the vector database for chunks whose vectors are closest to the query vector. This returns the top-K most similar chunks. The value of K depends on the use case. More chunks give the model more context but increase cost and the chance of including irrelevant material.

3. Re-ranking (optional): A secondary model scores the retrieved chunks by relevance to the specific question, reordering them so the most useful ones appear first. Re-ranking improves precision when the initial retrieval returns mixed results.

4. Prompt Construction: Assemble a prompt that includes the retrieved chunks and the user's question. The prompt instructs the model to answer based on the provided context and to cite sources.

5. Generation: The LLM generates a response grounded in the retrieved chunks. The answer is returned to the user, ideally with citations pointing back to the source documents.

Practical tip

The embedding model used for ingestion and the one used for queries must be the same. If you change your embedding model, you need to re-embed every document in your database. This is why choosing an embedding model early and sticking with it matters for production systems.

Step	Name	What Happens
1	Loading	Read the source document and extract text. PDFs need OCR or text extraction. HTML needs tag stripping. Tables may need special handling to preserve row-column structure.
2	Chunking	Split the document into smaller pieces (chunks) that fit within the model's context window. Chunk size and overlap determine retrieval granularity. A chunk that is too large dilutes relevance; one that is too small loses context.
3	Embedding	Convert each chunk into a vector (a list of numbers) using an embedding model. The vector captures the semantic meaning of the chunk. Similar content produces similar vectors.
4	Indexing	Store the vectors in a vector database along with the original text and metadata (source, page number, date). The database supports fast similarity search across millions of vectors.

Phase 1: Document Ingestion

Before you can retrieve documents, you need to process them into a format the retrieval system can search. The ingestion pipeline takes raw documents, PDFs, web pages, markdown files, spreadsheets, and converts them into searchable chunks stored in a vector database.

Phase 2: Query and Generation

When a user asks a question, the system retrieves relevant chunks and passes them to the LLM to generate an answer. This phase happens in real time, so latency matters.

Embeddings and Vector Databases

Embeddings are the foundation of RAG. An embedding model takes a piece of text and produces a vector, a sequence of numbers that represents the text's meaning. The key property is that texts with similar meanings produce vectors that are close together in the vector space. A chunk about refund policies and a question about how to get your money back will have similar vectors even though they share few words.

This is what makes semantic search possible. Traditional keyword search looks for exact word matches. If a document says cancellation policy and the user asks about refund terms, keyword search may miss the connection. Embedding-based search captures the meaning, so semantically related content surfaces regardless of vocabulary.

Embedding models come in different sizes and dimensions. A smaller model might produce 384-dimensional vectors; a larger one might produce 1,536-dimensional vectors. Higher dimensions can capture more nuance but require more storage and compute for similarity search. The choice depends on the tradeoff between precision and cost for your use case.

Vector Databases

A vector database stores embedding vectors and supports fast similarity search. When you query with a vector, the database finds the closest stored vectors using a distance metric, typically cosine similarity or Euclidean distance. The database returns the top-K results along with their associated text and metadata.

Several vector databases are commonly used. Pinecone, Weaviate, Qdrant, Milvus, and Chroma are purpose-built options. PostgreSQL with the pgvector extension is a popular choice for teams that already use Postgres and prefer a single database for both structured data and vectors. The choice of database affects scale, latency, and operational complexity, and the retrieval quality is largely determined by the embedding model and the chunking strategy rather than the database itself.

Go deeper

How similarity search works

Optional

Cosine similarity measures the angle between two vectors. If two vectors point in the same direction, their cosine similarity is 1.0 (maximum similarity). If they are orthogonal, similarity is 0. If they point in opposite directions, it is -1. In practice, text embeddings almost always produce positive cosine similarity values because embeddings live in a constrained region of the vector space. The database computes similarity between the query vector and every stored vector, then returns the highest-scoring ones. For large databases, approximate nearest neighbour (ANN) algorithms trade a small amount of precision for dramatic speed improvements, so search runs across millions of vectors in milliseconds.

Chunking Strategies

Chunking is the process of splitting documents into pieces that the retrieval system can search independently. It sounds simple, and at a basic level it is, but the chunking strategy has an outsized effect on retrieval quality. A chunk that is too large contains multiple topics, so the embedding averages across them and matches become less precise. A chunk that is too small loses context, so the retrieved text may not contain enough information for the model to answer the question.

Strategy	How It Works	Tradeoffs
Fixed-size chunks	Split text into chunks of N tokens (typically 256 to 1,024). Simple to implement. May cut sentences or paragraphs mid-way.	Easy and predictable. Poor at preserving semantic boundaries. A chunk that splits a sentence in two loses meaning.
Sentence or paragraph chunks	Split on natural boundaries, sentences, paragraphs, or sections. Each chunk contains a complete unit of thought.	Better semantic coherence. Variable chunk sizes complicate token budgeting. Long paragraphs may exceed desired size.
Overlapping chunks	Each chunk overlaps with the next by a fixed number of tokens (e.g., 50 to 100). Information at the boundary appears in both chunks.	Reduces information loss at boundaries. Increases storage and may cause duplicate retrievals. The overlap size needs tuning.
Document-aware chunking	Use document structure (headings, sections, tables, page breaks) to define chunk boundaries. Markdown headers, HTML tags, or PDF structure guide the split.	Best semantic coherence for structured documents. Requires parsing logic per format. Unstructured text falls back to simpler strategies.

There is no universally optimal chunk size. The right choice depends on the document type, the kinds of questions users ask, and the context window of the LLM. A common starting point is 512-token chunks with 50-token overlap, then tuning based on evaluation results. If retrieval is returning chunks that are almost but not quite relevant, try smaller chunks. If retrieved chunks lack enough context for the model to answer, try larger ones or increase the overlap.

Retrieval Quality: Search and Re-ranking

Retrieval is the part of RAG that determines whether the model gets useful context. If the retrieval system returns irrelevant chunks, the model will produce a confident answer based on the wrong information, which can be worse than having no context at all. Retrieval quality is where the majority of RAG engineering effort goes.

Hybrid Search

Pure vector search captures semantic similarity but can miss exact matches. If a user searches for a specific product code or an exact phrase, vector search may return semantically related content that does not contain the exact term. Keyword search (like BM25) excels at exact matches but misses semantic connections.

Hybrid search combines both. It runs vector search and keyword search in parallel, then merges the results. The merge can use a weighted combination or a reciprocal rank fusion algorithm that balances the two score sets. Hybrid search consistently outperforms either method alone across diverse query types, which is why production RAG systems almost always use it.

Re-ranking

Re-ranking is a second-pass filter. After retrieval returns the top-K candidates (say, 20 chunks), a re-ranker model scores each chunk for relevance to the specific question and reorders them. The re-ranker is typically a cross-encoder model that takes the question and each chunk as a pair and produces a relevance score. Cross-encoders are more accurate than the bi-encoder used for initial retrieval, but they are also more expensive, so you only run them on the top candidates.

The improvement can be substantial. A common pattern. Retrieve 20 chunks with vector search, re-rank them, and pass the top 5 to the LLM. The re-ranker pulls the genuinely relevant chunks to the front and drops the noise, so the model gets a cleaner, more focused context window.

Query Transformation

Users do not always phrase their questions in ways that retrieve well. A user might ask Can I cancel my order after it ships? when the relevant document uses the term order modification rather than cancellation. The embedding may still catch the semantic connection, but a transformed query can do better.

Query transformation techniques include rewriting the query using an LLM to expand or rephrase it, generating multiple variations of the query and retrieving for each (multi-query retrieval), and extracting keywords from a conversational query that references earlier context. For example, in a chat where the user first asks about order 12345 and then asks when will it arrive, the retrieval system needs to understand that it refers to order 12345. A query transformation step can rewrite the second query as when will order 12345 arrive before searching.

Generation: Grounding the LLM

Once you have retrieved the relevant chunks, you need to construct a prompt that tells the model to use them. The prompt structure matters because the model needs clear instructions about what to do with the retrieved text.

Typical RAG prompt

You are a helpful assistant. Answer the user's question using only the information provided in the context below. If the context does not contain enough information to answer, say you do not know. Cite the source document for each claim.

<context> [retrieved chunks here] </context> Question: [user's question] Answer:

Several design choices in this prompt deserve attention. The instruction to use only the provided context tells the model to treat the retrieved chunks as its sole source of truth, which reduces the chance of the model falling back on its training data and potentially contradicting the context. The instruction to say you do not know when the context is insufficient gives the model permission to admit ignorance rather than fabricate. And the citation instruction encourages traceability, so the user can verify which document the answer came from.

The Lost-in-the-Middle Problem

As discussed in the prompt engineering guide, models pay uneven attention across long contexts. Information at the beginning and end of the prompt gets more weight than information in the middle. In

a RAG system with many retrieved chunks, this means chunks placed in the middle of the context window may be effectively ignored.

Two approaches help. First, keep the number of retrieved chunks small. Five well-chosen chunks are better than twenty mixed ones, both for attention quality and for cost. Re-ranking helps here by ensuring the most relevant chunks are the ones you include. Second, order the chunks strategically. Place the most relevant chunk first and the second most relevant last, so both the beginning and end of the context carry the strongest signal.

Handling Conflicting Sources

In practice, retrieved documents sometimes contradict each other. An older version of a policy says one thing; a newer version says another. Two support articles give different instructions for the same problem. The model needs guidance on how to handle this. Options include instructing the model to prefer the most recent source (which requires including timestamps in the context), telling the model to note the conflict and present both perspectives, or filtering at the retrieval stage to exclude outdated documents using metadata.

Evaluating RAG Systems

A RAG system has two components that can fail independently. retrieval (did you find the right documents?) and generation (did the model use them correctly?). Evaluating a RAG system means measuring both, because good retrieval with bad generation produces wrong answers, and bad retrieval with good generation produces answers to the wrong questions.



Pascal Academy

Pascal Academy's RAG engineering course covers evaluation frameworks, retrieval optimisation, and production deployment patterns. Reach out to learn about cohort programmes for your engineering team.

Ask about team training

Metric	What It Measures	How to Evaluate
Context relevance	Did the retrieval system return chunks that actually contain the answer? Measures retrieval quality independent of generation.	Human review of retrieved chunks for a sample of queries, or an LLM-as-judge that scores whether each retrieved chunk is relevant to the question.
Groundedness	Is the model's answer supported by the retrieved context? Every claim in the answer should be traceable to a specific chunk.	Check whether each sentence in the answer can be attributed to a retrieved chunk. An LLM-as-judge can verify this by comparing the answer against the context.
Answer relevance	Does the answer actually address the user's question? A grounded answer that does not answer the question is still a failure.	Human review or LLM-as-judge scoring whether the answer addresses the specific question asked.
Recall at K	Measures the percentage of relevant chunks in the database that the retrieval system found in its top-K results.	Requires a labelled set of questions with known-correct source documents. Measure what percentage of correct sources appear in the top-K retrieved chunks.

Building an evaluation set is the single most valuable investment for a RAG system. Collect 50 to 100 real user questions with known correct answers. For each, note which document and chunk contains the answer. Run your pipeline against this set and measure the metrics above. When you change chunking strategy, embedding model, or prompt structure, re-run the evaluation. This converts RAG development from guesswork into measurable improvement.

RAG vs Fine-Tuning vs Long Context

Three approaches exist for giving a model access to information beyond its training data, and they are often confused. Understanding when to use each is a practical decision that depends on the use case.

Approach	How It Works	Best For
RAG	Retrieve relevant documents at query time and include them in the prompt. Knowledge lives in an external database and updates without retraining.	Frequently changing data, large knowledge bases, use cases requiring citations, and when you need to trace answers to sources.
Fine-tuning	Train the model further on domain-specific data so the knowledge is baked into the weights. Updates require retraining.	Teaching the model a consistent style or format, domain vocabulary, or reducing prompt length. Less suited for factual knowledge that changes.
Long context	Put the entire document or knowledge base directly in the prompt. No retrieval system needed. Works when the data fits in the context window.	Small, fixed document sets. Analysis of a single large document. Quick prototyping before building a retrieval pipeline.

These approaches are complementary. A production system might fine-tune a model for tone and format, use RAG for factual knowledge that changes, and use long context for analysing a specific large document the user uploads. The decision is driven by how often the data changes, how much data there is, and whether you need citation traceability.

A practical heuristic. If your knowledge base is under 100,000 tokens and rarely changes, long context is simpler and avoids the infrastructure cost of a vector database. If the knowledge base is large or changes frequently, RAG is the right choice. Fine-tuning is appropriate when the problem is style or vocabulary rather than knowledge.

Common Failure Modes

Retrieval Failure

The retrieval system returns chunks that do not contain the answer. The model then either hallucinates based on its training data or says it cannot find the answer. Retrieval failure has several causes. Poor chunking that splits the relevant information across chunks, an embedding model that does not capture the domain well, or a query that is phrased differently from the document text. Debugging retrieval requires examining what chunks were returned for a failing query and comparing them to the chunks that should have been returned.

Context Overload

The retrieval system returns too many chunks, and the relevant information gets diluted. The model has to process a large context, which increases cost and latency, and the lost-in-the-middle effect means some chunks get ignored. The fix is to reduce the number of retrieved chunks, use re-ranking to surface the most relevant ones, and keep the total context within a reasonable token budget.

Stale Knowledge Base

The knowledge base contains outdated documents, and the model answers based on old information. A policy document updated last month coexists with the previous version from two years ago, and the retrieval system returns both. The model may pick the older one or present conflicting information. The fix is to implement versioning and cleanup. Remove or archive old documents, include timestamps in metadata, and filter retrieval by date when recency matters.

Over-Reliance on Retrieved Context

When the retrieved chunks are wrong or misleading, the model faithfully produces a wrong answer. RAG reduces hallucination by grounding the model, but grounding to the wrong source is just as harmful. The model trusts the retrieved context because the prompt tells it to, so the quality of the retrieval system directly determines the quality of the output. There is no prompt-level fix for bad retrieval.

Agentic RAG

Everything in this guide so far describes what is sometimes called vanilla RAG. The user asks a question, the system retrieves chunks once, and the model generates an answer from whatever came back. That works well for straightforward lookups where the answer lives in a single document. It breaks down when a question requires information from multiple sources or when answering it requires following a chain of references.

Consider a question like, what server does Project X use, and what are its specifications. A vanilla RAG system finds documents about Project X, which mention a server ID. It retrieves those documents and hands them to the model. The model sees the server ID but has no way to look up what that ID corresponds to in a separate hardware database. The answer is incomplete because the retrieval was a single pass, and the system had no mechanism to take the server ID and run a second search.

Agentic RAG addresses this by making retrieval iterative and goal-directed. Instead of a single retrieve-then-generate step, the system plans a search strategy, executes queries, evaluates whether it has enough information to answer, and if not, runs additional searches. The word agentic refers to the system having agency over its own retrieval process rather than following a fixed retrieve-and-generate script.

Agentic RAG systems are typically built as a multi-agent pipeline where different components handle different roles. Think of it as a research department rather than a single search engine. Each agent has a specialised job, and the output of one feeds into the next.

Go deeper

What the sufficient context agent checks

Optional

The sufficient context agent evaluates three things before allowing generation to proceed. First, it reads the retrieved snippets to see if they contain the information needed to answer the question. Second, it reviews a draft answer against the original prompt and the snippets to check for completeness. Third, and this is the part that distinguishes it from a simple completeness check, it identifies exactly what is missing and generates specific feedback for the next retrieval round. Rather than just saying the context is insufficient, it produces a targeted instruction like search for adverse events or allergic reactions in clinical notes. This targeted feedback is what makes the iteration efficient rather than random.

Summary

RAG gives language models access to external knowledge at query time by retrieving relevant documents and including them in the prompt. The ingestion pipeline processes documents into chunks, embeds them, and stores them in a vector database. At query time, the user's question is embedded, similar chunks are retrieved, and the model generates an answer grounded in them. Retrieval quality, driven by chunking strategy, embedding model choice, hybrid search, and re-ranking, determines whether the model gets useful context. The generation prompt controls how the model uses that context. Evaluating a RAG system means measuring both retrieval (context relevance, recall) and generation (groundedness, answer relevance) independently. RAG complements fine-tuning and long-context approaches, and the right choice depends on how often the data changes and how much there is. For questions that span multiple sources or require chained lookups, agentic RAG extends the pipeline with iterative retrieval, query planning, and a sufficient-context check that catches gaps before the model generates its answer.

Agent	Role
Orchestrator	Evaluates the user's request and decides whether a single retrieval pass is enough or whether the query needs to be decomposed into multiple sub-queries. Delegates work to specialised agents.
Planner	Maps out which information sources to consult and in what order. If the question spans finance and project management, the planner decides to check the finance database first, then the project logs.
Query Rewriter	Translates the user's original question into optimised search queries. A question like what is up with Project X becomes targeted queries like Project X Q3 status report and Project X key blockers. This improves retrieval accuracy because the rewritten queries match document language more closely.
Search Agent	Executes the refined queries against one or more retrieval sources. In cross-corpus setups, this agent also decides which database to search based on the planner's routing.
Sufficient Context Agent	Reviews the retrieved snippets and a draft answer to determine whether the model has everything it needs. If the user asked about medications, diet, and allergies but the retrieved context only covers medications and diet, this agent flags the gap and sends the system back for another round of retrieval.
Synthesis Agent	Once the sufficient context agent confirms the retrieved information is complete, this agent generates the final response, grounded in all the collected context.

When to Use Agentic RAG

Agentic RAG adds complexity and latency. The multi-agent pipeline requires more LLM calls, more retrieval rounds, and more orchestration logic than vanilla RAG. For simple lookup questions where the answer lives in a single document, this overhead is wasted. The vanilla pipeline retrieves once and answers, which is faster and cheaper.

Agentic RAG earns its cost when questions are multi-hop, meaning they require chaining information across sources. A question like what are the specs of the server used in Project X requires finding the project document, extracting the server ID, then searching a hardware database for that ID. A single retrieval pass cannot solve this because the second search depends on information found in the first. Agentic RAG handles this naturally because the planner decomposes the question, the search agent executes each sub-query, and the iteration loop continues until the sufficient context agent confirms all pieces are present.

Agentic RAG also helps when information is spread across separate databases managed by different teams. A large organisation may have HR records in one system, finance data in another, and project documentation in a third. Cross-corpus retrieval, where the planner routes queries to the right database, is a natural fit for agentic RAG. Google Research reported that their agentic RAG framework achieved 90.1 percent accuracy on multi-hop questions even when the system had to select the correct corpus from four options, with latency within 3 percent of single-corpus retrieval.

The part that makes agentic RAG different from vanilla RAG is the sufficient context check. In a standard system, retrieval happens once and the model generates regardless of whether the retrieved context actually contains the answer. The model may hallucinate to fill the gap, or it may say it cannot find the information. Both outcomes are bad. The sufficient context agent adds a quality-control step that catches gaps before generation and triggers additional retrieval passes to fill them.

About this guide

This guide is part of Pascal Academy's AI Fundamentals series, covering LLMs, prompt engineering, RAG, agents, and context windows. The full series and hands-on courses are available at Pascal Academy. For teams looking to upskill, we offer custom cohort programmes tailored to your stack and use cases.

Frequently asked questions

What is RAG in AI and how does it work?

RAG stands for Retrieval-Augmented Generation. It is a technique where an AI model retrieves relevant documents from a knowledge base before generating a response. Instead of relying only on its training data, the model pulls in external information, grounds its answer in that information, and cites the source.

How does RAG improve AI outputs for finance?

RAG allows the model to reference actual filings, transcripts, and research documents instead of relying on what it learned during training. This produces more accurate, current, and specific outputs. The retrieval step also creates a citation trail so you can verify where each claim came from.

What is Agentic RAG and how does Pascal implement it?

Agentic RAG is an advanced form of RAG where the system iteratively retrieves information, evaluates whether it has enough context, and goes back for more if needed. Pascal's research agents use this approach through an orchestrator that plans queries, retrieves from filings and your internal documents, and synthesizes outputs with citations tracing back to the source line.

What is the difference between RAG and fine-tuning?

RAG retrieves external documents at query time to ground the model's response. Fine-tuning retrains the model on new data to change its behavior. RAG is better for keeping information current and producing cited outputs. Fine-tuning is better for adapting tone or domain expertise. They are not mutually exclusive.